Rethinking What You Already Know

Basic Scripting — a live demo

Thursday 2026-09-24, 10:00 AM CT · 45 minutes + 15 Q&A

italic = you do NOT say this · **roman = say it out loud**

```
DECK KEYS
→ / space  advance    ←  back one
Home  first slide    End  last slide
S  speaker window (put it on the big monitor)
F  fullscreen   B  black   G  go to slide   R  reset timer

Slide 22 is the PowerShell reserve — only if asked.
```

1 Basic Scripting

italic = don't say it · **roman = say it**

WHERE YOU ARE

Title slide. You have not touched the terminal yet.

Front matter first, then the poll. Do not rush to the terminal.



SAY

THE OPEN. Slow, warm, unhurried. This is the only part of the hour where you are asking them to trust you rather than showing them something.

"Hi, I'm Chad Miller. Today I want to talk about a two-line script."

"It works. That's the problem."

Let that sit for a second. Then reframe it as THEIR problem -- this is the sentence for the two VPs who have never met you:

"What I actually want to talk about is the gap between IT RAN and IT'S CORRECT. And who owns that gap once the script leaves the person who wrote it."

THE WORD -- PROMISED, NOT GIVEN. It arrives on the next slide, after the terminal.

"There is a word for the property, and you probably already know it. We are going to be missing it all session."

THE WITHHOLD -- deliberate. Do not say the word yet. They get it later, after they have watched it not happen. It lands harder with the evidence already on the table.

"I will give you the word in a few minutes. I would rather you watch it go missing first."

⛔ *NOTHING ELSE ABOUT THE WORD HERE. No definition, no examples, and do NOT say the word itself. If you say it now, the slide after the terminal has nothing to land.*

NOW the poll. You have earned it.

"Show of hands -- who here has edited a hosts file?"
(wait)
"Keep your hand up if you've done it at two in the morning."

Then the contract, plainly:
"This is going to be basic. Deliberately. I'm going to show you something inside it anyway."

THEN SET THE BLAST RADIUS. Do not skip this -- it is the whole frame.
"One more thing before we start. Everything I show you today runs on ONE machine in front of us. And it is a copy, not my real hosts file -- editing that live on a webinar seemed like a bad idea."

Then the turn -- this is the frame for the whole hour:

"But picture it running unattended on four thousand. Every decision we make in the next hour changes meaning at that scale."

Throwaway delivery. Half a shrug, like it just occurred to you. Do NOT underline it or pause after it -- it works because you move on.

"Which, now that I say it out loud, is the entire talk."

ON THE WAY OUT →

DO NOT ADVANCE. Alt-tab to the terminal and stay there for nine minutes.
This slide holds while you work. Everything below is the Tier 1 block.

TALK TRACK FOR EVERY COMMAND. Never type in silence -- say what it is BEFORE you hit enter.

```
./reset.sh
```

"First thing, I am going to reset my environment."

"I will be making a lot of deliberately crazy changes to the hosts file as we go -- and nobody wants to watch me hand-fix something I broke on purpose."

"So that puts everything back to a known state."

```
cat hosts
```

"Now let us look at the file."

"This is a real hosts file. Comments, multi-hostname lines, somebody's override sitting in the middle."

"It is not the hosts file on this machine. But it is a genuine one -- drop it in the right place and it would work exactly as it sits."

Let them look. This is the last calm moment before anything breaks.

```
./resolve.sh                  -> 10.0.0.5   (line 6 -- FIRST match wins)
```

"And this one is mine too. Not a system tool."

"It simulates what the OS resolver does. It reads that sandboxed file, applies the same rule the operating system applies, and tells me which entry actually wins."

"It only ever looks up the one host we care about -- so there is nothing for me to type, and nothing for me to get wrong."

"And the rule is the whole reason we are here: first match wins. The first line that names the host is the answer. Everything under it is dead weight, no matter how recently somebody added it."

DEFINE THE TIERS. They are about to see the word on a filename -- say this first.

"One piece of vocabulary before we look at it, and it is mine -- this is not a standard, so nobody has to pretend they have heard of it."

"I am going to show you three versions of this script. I am going to call them Tier One, Tier Two and Tier Three."

"Tier One is what everybody writes when the ticket is open."

"Tier Two is the fix you reach for when Tier One embarrasses you."

"And Tier Three is what it actually takes."

Then, lightly -- it lowers the stakes and it is true:

"That is the whole scale. It is just how I am going to keep them straight."

```
cat tier1.sh
```

"So here is the script. Two lines. And I want to be fair to it -- this is not a strawman. This is what I would write at two in the morning, with a ticket open and somebody asking me for the fourth time if it is

done yet."

```
./tier1.sh                    -> Backup server added successfully.
```

"It ran. It says it worked. Exit code zero, green check, go back to bed... or the party."

```
./resolve.sh                  -> 10.0.0.5    UNCHANGED
```

STOP TALKING AFTER THAT LAST LINE. Let them read it.
Then: "The script didn't fail. It succeeded at doing nothing. Your backups are STILL writing to a decommissioned box."

"And your monitoring will go red. Not tonight -- whenever that backup is actually needed."

Then straight at the room -- this is the line for three VPs:

"And who gets paged?"

Let them answer it in their heads. Then:

"It might not be whoever wrote that script at two in the morning. It lands on whoever is holding the pager."

"And a two-line change to a script that edits a hosts file, from weeks ago, is not obviously connected to a backup failing tonight. You and I know that is the problem. The person holding the pager has no reason to."

Then ask it like a real question, not a rhetorical one:

"Where is that documented, exactly?"

Let the silence do it. Then one word:

"Nowhere."

Move straight on. Do NOT explain it -- it gets answered much later.

Make it undeniable -- TALK FIRST, then type:

"So it did not do what I expected. Now -- before I go on a bug hunt, before I read a single line of it -- what is the first thing I am going to do?"

Let them answer. Somebody will say it. If nobody does, say it yourself:

"Run it again. See if it takes the second time. Everybody does this. I do this."

Then, flat:

"This script is not safe to run a second time. Watch."

```
./tier1.sh; ./tier1.sh; ./tier1.sh
```

```
grep -c backup-server hosts   -> 5
```

"Five entries for one host."

```
./resolve.sh                  -> 10.0.0.5. still.
```

WHY THIS FILE. Say it here, once.

"And I picked this file on purpose."

"This is slash-etc-slash-hosts on Linux and macOS. On Windows it lives under System-thirty-two, drivers, etc."

"Same file. Same format. Same rules."

"No setup, no platform argument, nothing to install. Every one of you has this file on the machine in front of you right now."

THE ASIDE -- and this is your exit. Start alt-tabbing back to the browser as you say it. The words cover the move, so it reads as staging instead of fumbling. Fifteen seconds. Do NOT expand it.

"And a hosts file takes the FIRST match. Plenty of config formats take the LAST one -- in which case your duplicate silently wins and you never find out you had two."

You should be looking at the deck by now. Say the last line, and advance ON it.

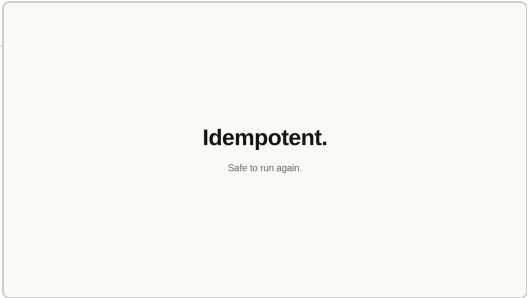
"Either way, you are left with a file nobody can reason about."

The next slide is the word.

2 Idempotent.

target 0:10

italic = don't say it · **roman = say it**



WHERE YOU ARE

The word is on screen, alone. You have just said the colon line and taken a beat.

You are back from the Tier 1 terminal block. They have watched a script succeed at doing nothing. NOW the word means something -- they have already seen it missing.

SAY

Flat. Do not sell it. The screen says it; you explain it.

"A script is idempotent if running it twice leaves the machine exactly the way running it once did. Run it a hundred times, same result. Nothing accumulates, nothing drifts."

Then make it theirs, not a vocabulary lesson:

"It is not an academic idea. It is what Ansible and Terraform and every config management tool you have ever bought sells itself on. It is the reason you can run the playbook again without holding your breath."

"And it is the single most common thing missing from a script somebody wrote in two minutes to solve a problem that was on fire."

Then, pointing back at the terminal:

"What you just watched was not a script that failed. It was a script that was not safe to run again, run again."

ON THE WAY OUT →

> *Advance into the obvious fix.*

3 The problem

target 0:12

The problem

```
echo "10.0.0.9  backup-server" >> /etc/hosts
```

italic = don't say it · **roman = say it**

WHERE YOU ARE

You are back from the terminal. They have just watched Tier 1 fail silently. Only the append is on screen.

Nothing on screen says his lines. The fix is NOT marked red.

SAY

"Okay. So we've got duplicates. Everybody sees the fix, right?"

Let someone in the room say it. They will.

CHEAT SHEET -- what the next slide shows: change the DOUBLE arrow to a SINGLE arrow. One character.

ON THE WAY OUT →

> *Advance -- this is a build. The append stays put and the fix appears underneath it.*

4 The solution

target 0:12

The solution

```
echo "10.0.0.9  backup-server" >> /etc/hosts
```

```
echo "10.0.0.9  backup-server" >  /etc/hosts
```

italic = don't say it · **roman = say it**

WHERE YOU ARE

The fix is on screen under the append. Nothing marks it as wrong -- on purpose. Play it straight.

SAY

"One character."

Sincere. You are selling the fix, not winking at it.

ON THE WAY OUT →

> *Advance -- the full hosts file, before the fix runs.*
>
> *DO NOT demo this live. Everyone knows what > does; three live minutes proving it is the one place this session can drag.*

5 Before

target 0:13



italic = don't say it · **roman = say it**

WHERE YOU ARE

The full hosts file is on screen, BEFORE the one-character fix. It is the same file they just watched in the terminal, so nothing here is new to them.

SAY

"Here's the file before we run the fix."

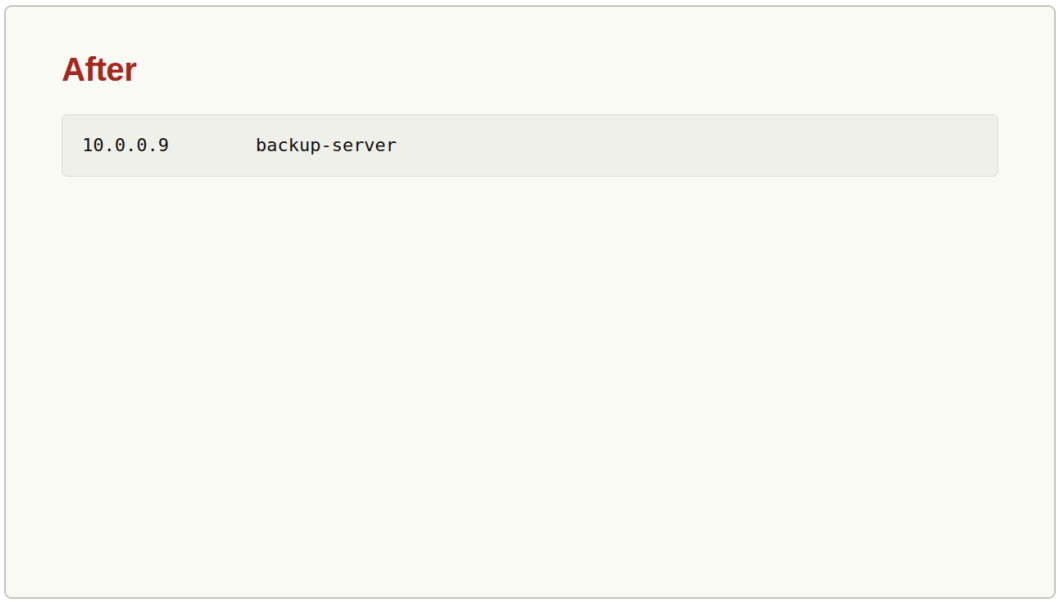
Give them two seconds with it. Do not walk the lines -- that is two slides from now, and pointing at the override here gives the ending away.

ON THE WAY OUT →

> *Advance. The next slide is the same file after the fix, and it is almost empty.*

6 After

target 0:13



italic = don't say it · **roman = say it**

WHERE YOU ARE

One line on screen. That is the whole file now.

SAY

Say nothing. Let the empty space do it.

"Duplicates: solved."

Deadpan. This is the payoff -- nothing on screen has said it.

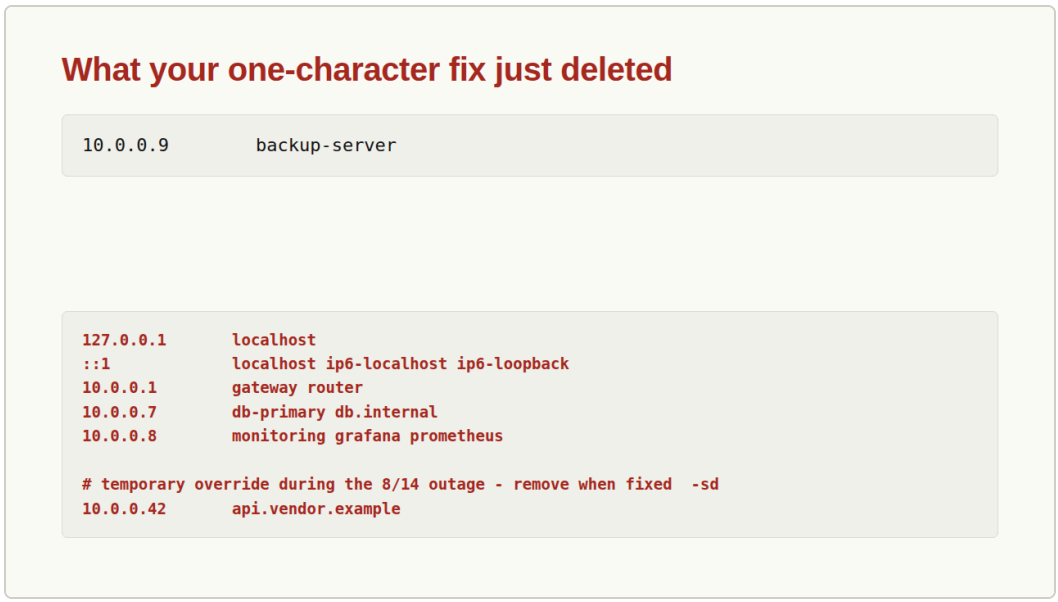
"And that is the Tier Two error. It is not carelessness. It is the thing we do when Tier One fails."

ON THE WAY OUT →

> *Advance -- this is a build. The one line stays put and everything that single character deleted comes in underneath it, in red.*

7 What your one-character fix just deleted

target 0:14



italic = don't say it · **roman = say it**

WHERE YOU ARE

The deleted lines are on screen in red.

SAY

Read them out loud, slowly. Especially the last two.

"That last one is somebody's outage override. It has their initials on it."

"S-D is a person with a stake in the health of this machine. And we just erased all trace that they were ever there, and that they ever fixed anything."

"This is the tier where the room goes quiet, because most of us have done exactly this."

Say that as a confession, not an accusation. You have done it too.

MENTION ONLY. Do NOT build this out -- one breath, then move.

"There is a better version of this, and some of you are already thinking it: grep out the line, write it to a temp file, move it back. Delete, then append."

"That is better. Right up until the pattern also eats backup-server-old."

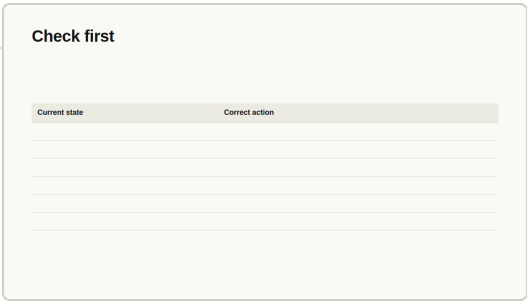
ON THE WAY OUT →

> *That last sentence is your bridge -- every quick fix so far has broken something. Advance to the state-machine table.*

8 Check first

target 0:15

italic = don't say it · **roman = say it**



WHERE YOU ARE

Empty table on screen -- header row only, every cell hidden. Nothing is given away.

They have now watched three fixes break something. This is the turn from patching to thinking, and the table is built WITH them, one row at a time.

SAY

SET IT UP FIRST. They are looking at an empty table and need to know what it is.

"Alright. Three attempts at this script. Three different ways to break the hosts file."

Now hand them the fix they are already reaching for, and take it seriously -- this is the right instinct and you are not going to mock it.

"And I know what you are thinking, because I thought it too. Check first. Is the host name already in the hosts file? If it is, do not add it again."

Agree with them before you outgrow it.

"That is the right instinct."

Then the turn. Do NOT rush this -- it is the pivot of the whole session.

"And it is still a two-line script."

"But every fix today has been one more line bolted onto the same script. And each one just moved where it breaks."

"So let us stop editing the script, and ask a different question."

Gesture at the empty table. THIS question is the table.

"Check for what?"

"Because every answer to that is a state the hosts file can be in. And every state needs its own right answer. So we are going to build that list together."

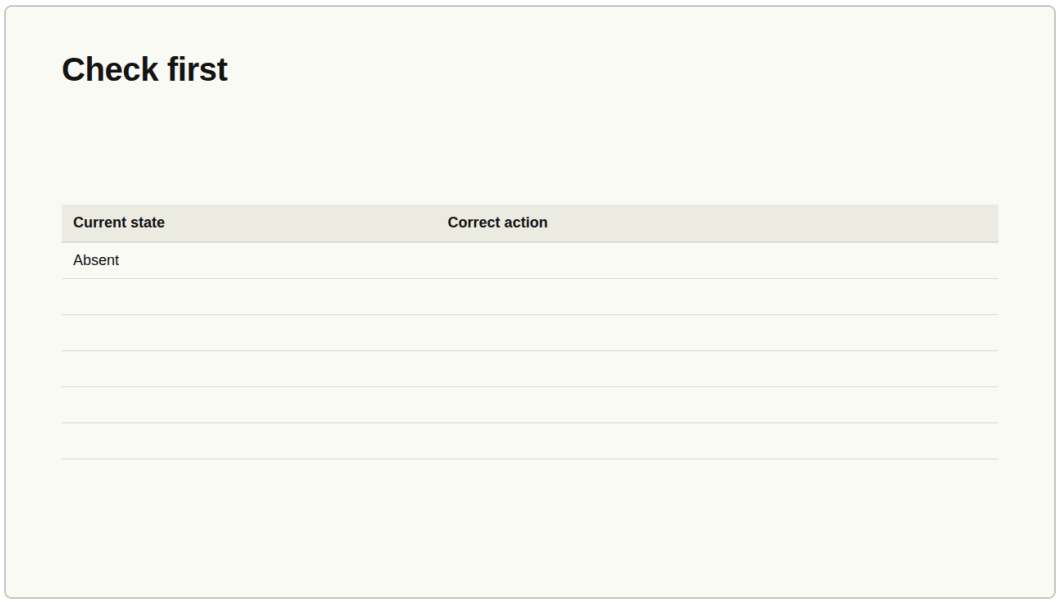
Do NOT say the words STATE MACHINE yet. The room earns it at the bottom of the table.

ON THE WAY OUT →

> *Advance -- the first state appears, with its action hidden.*

9 Check first

target 0:15



italic = don't say it · **roman = say it**

WHERE YOU ARE

New state on screen, action hidden: Absent.

SAY

"The host name is not in the hosts file at all. What do we do?"

Let them answer before you advance.

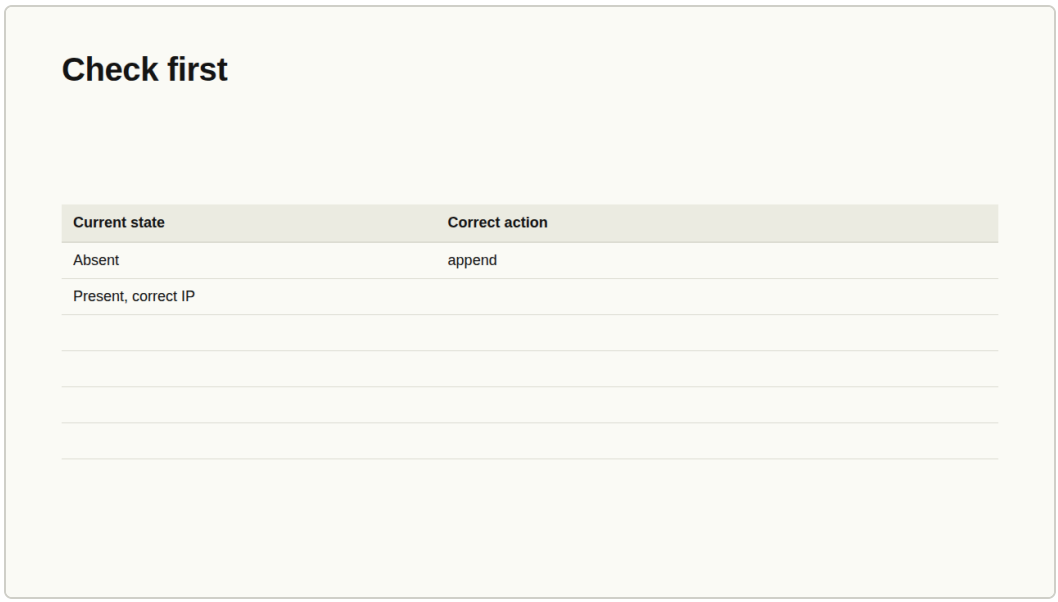
CHEAT SHEET -- the answer this row reveals: APPEND

ON THE WAY OUT →

> *Advance -- reveals the answer and the next state.*

10 Check first

target 0:15



italic = don't say it · **roman = say it**

WHERE YOU ARE

The answer to the last one just appeared: Absent -> append.

New state on screen, action hidden: Present, correct IP.

SAY

CONFIRM THE ANSWER THAT JUST APPEARED -- out loud, before the new question.

"Append it, right. If the host name is not in the hosts file at all, it is safe to add it. Everybody good with that?"

"Okay, now our next case: the host name IS in the hosts file, and the address on that line is already the one we want. What do we do?"

Let them answer before you advance.

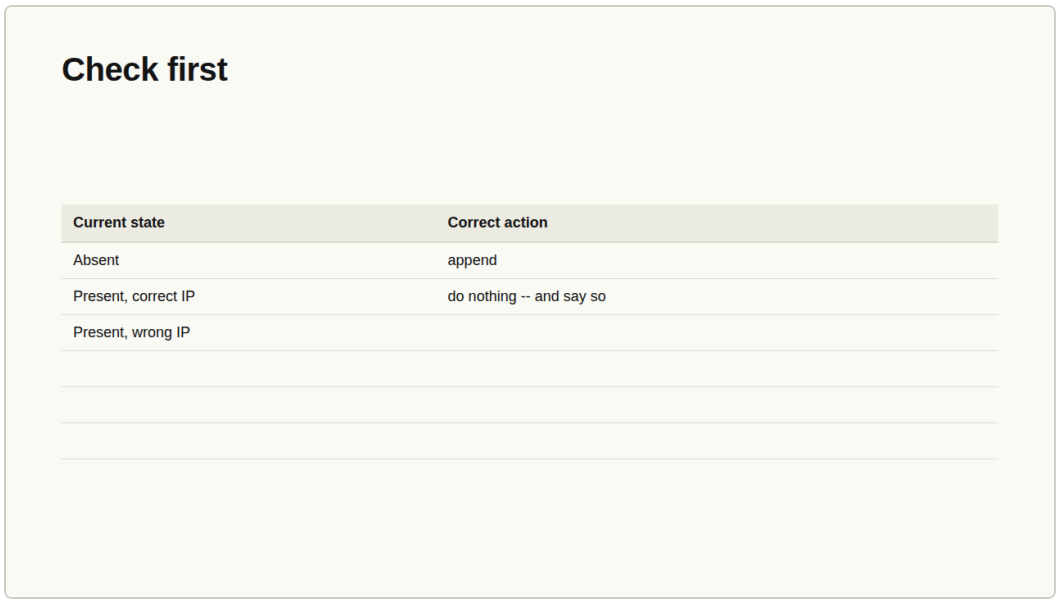
CHEAT SHEET -- the answer this row reveals: DO NOTHING -- AND SAY SO

ON THE WAY OUT →

> *Advance -- reveals the answer and the next state.*

11 Check first

target 0:15



italic = don't say it · **roman = say it**

WHERE YOU ARE

The answer to the last one just appeared: Present, correct IP -> do nothing -- and say so.

New state on screen, action hidden: Present, wrong IP.

SAY

CONFIRM THE ANSWER THAT JUST APPEARED -- out loud, before the new question.

"Do nothing -- right. But look at what I wrote next to it: AND SAY SO. In a log, silence and success look exactly the same."

"The host name is in the hosts file, but the address on that line is the OLD one. Now what?"

Let them answer before you advance.

CHEAT SHEET -- the answer this row reveals: REPLACE, DON'T APPEND

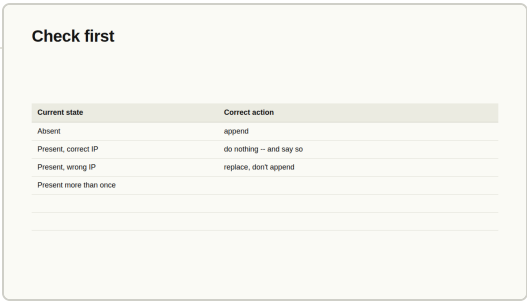
ON THE WAY OUT →

> *Advance -- reveals the answer and the next state.*

12 Check first

target 0:15

italic = don't say it · **roman = say it**



WHERE YOU ARE

The answer to the last one just appeared: Present, wrong IP -> replace, don't append.

New state on screen, action hidden: Present more than once.

SAY

CONFIRM THE ANSWER THAT JUST APPEARED -- out loud, before the new question.

"There it is. Replace, not append."

Then the callback.

"And notice what just happened."

"That is the case check-first gets wrong. It asks whether the HOST NAME is in the hosts file. And it is -- sitting there with the wrong address. So it skips."

"Which puts us back at Tier One. Failing quietly, with extra steps."

"The host name is in the hosts file twice, on two lines, with two different addresses. Which one do we keep?"

CHEAT SHEET -- the answer this row reveals: REFUSE -- YOU CANNOT KNOW WHICH IS AUTHORITATIVE

Nobody volunteers REFUSE. They will argue about which line to keep. Let them.

When the discussion has run its course:

"Listen to what is happening."

"Here we all are, in the same session, looking at an identical and common problem we are all familiar with -- and we cannot agree on which one is the right answer."

Then land it.

"Neither can a script."

"There is no way to know which one is authoritative. So the only honest thing it can do is refuse -- and say why."

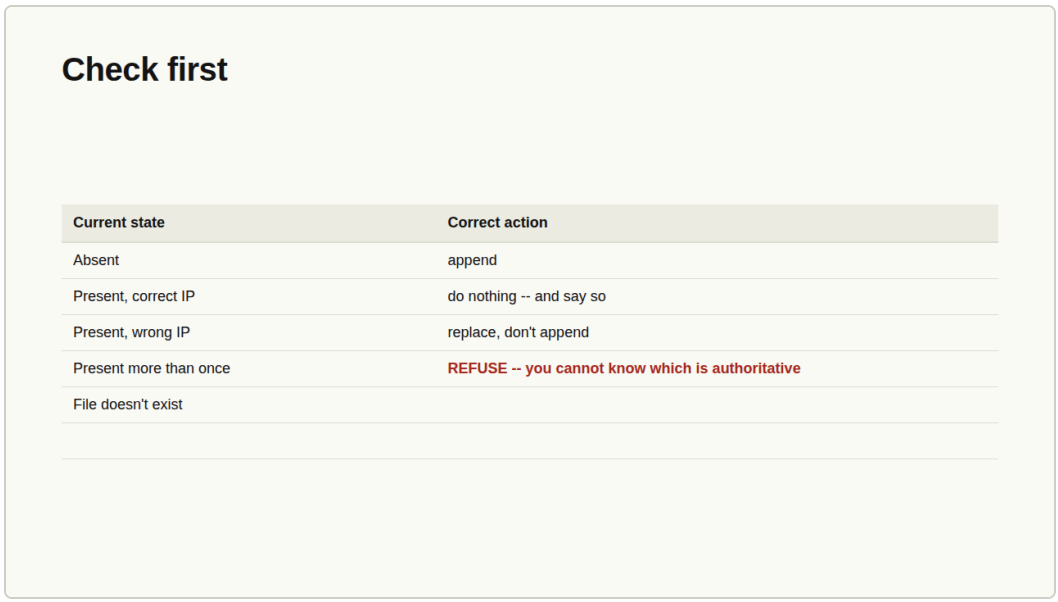
Now advance.

ON THE WAY OUT →

> *Advance -- reveals the answer and the next state.*

13 Check first

target 0:15



italic = don't say it · **roman = say it**

WHERE YOU ARE

The answer to the last one just appeared: Present more than once -> REFUSE -- you cannot know which is authoritative.

New state on screen, action hidden: File doesn't exist.

SAY

CONFIRM THE ANSWER THAT JUST APPEARED -- out loud, before the new question.

"Refuse. And that one took us a minute, which is the point."

"There is no hosts file at all. Now what?"

Let them answer before you advance.

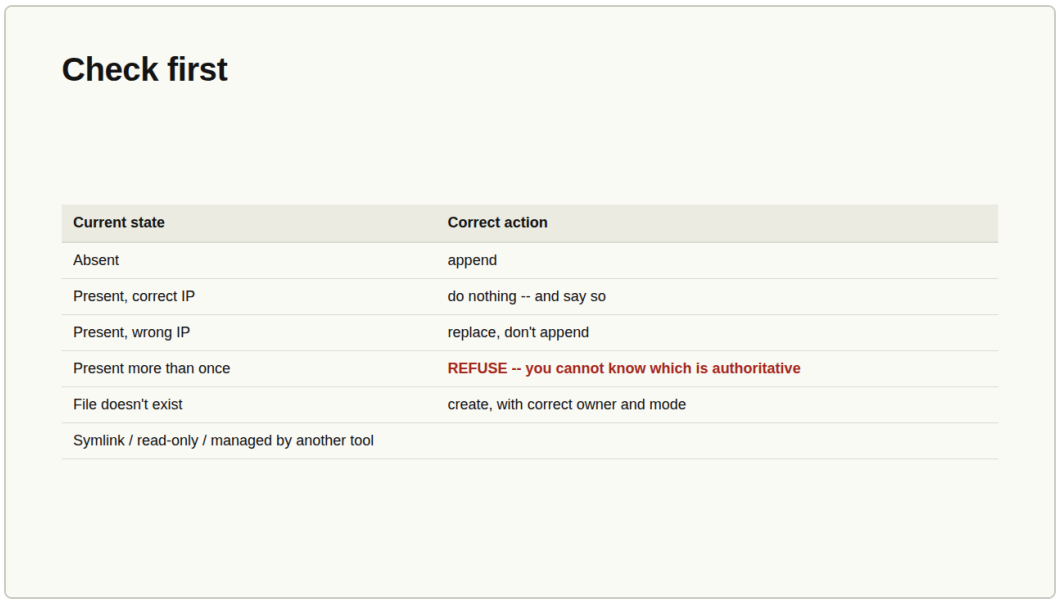
CHEAT SHEET -- the answer this row reveals: CREATE, WITH CORRECT OWNER AND MODE

ON THE WAY OUT →

> *Advance -- reveals the answer and the next state.*

14 Check first

target 0:15



italic = don't say it · **roman = say it**

WHERE YOU ARE

The answer to the last one just appeared: File doesn't exist -> create, with correct owner and mode.

New state on screen, action hidden: Symlink / read-only / managed by another tool.

SAY

CONFIRM THE ANSWER THAT JUST APPEARED -- out loud, before the new question.

"Create it. With the right owner and the right mode, because a hosts file nobody can read is its own outage."

"The hosts file is a symlink. Or it is read-only. Or Puppet owns it. What now?"

Let them answer before you advance.

CHEAT SHEET -- the answer this row reveals: REFUSE, LOUDLY

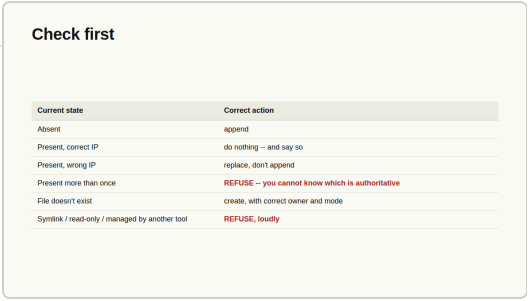
ON THE WAY OUT →

> *Advance -- reveals the answer. That completes the table.*

15 Check first

target 0:15

italic = don't say it · **roman = say it**



WHERE YOU ARE

The whole table is on screen. The last answer just appeared: symlink / read-only / managed -> REFUSE, loudly.

SAY

The whole table is up. THIS is where the room earns the word -- it was deliberately withheld on the empty table.

CONFIRM THE LAST ANSWER FIRST -- same as every other row.

"Refuse. Loudly. If something else owns that file, you do not even have the option of guessing -- you cannot alter the file."

"That is a pretty big problem, and it is going to take a human to fix it. Right?"

Now step back and let them look at the whole thing.

"So look at what we just built. That is not a check."

Try to get it from the room. If nobody says it, say it yourself and move on -- do not fish.

"That is a state machine."

"Six states. Every one of them has a correct action, and the correct action is different for each."

Then point at both red rows. Do not explain them yet -- the terminal does that.

"And two of them -- that is one third of the possible outcomes of this script -- are to REFUSE TO ACT."

ON THE WAY OUT →

DO NOT ADVANCE. The table stays on screen for the whole terminal block -- it is what they reason against. Alt-tab now.

```
cat tier3.sh
```

"This is our Tier 3 script. Our state machine. Let us go through it."

Scroll it slowly. Do NOT read it line by line -- some of them read bash and some of them do not. Narrate the shape.

"Same job as before. Same two lines of real work buried in the middle of it."

"Before it touches anything, it asks two questions about the file itself. Is this a symlink? Can I actually write to it? If either answer is wrong, it stops right there and says so. It does not try and hope."

"Then it counts. How many lines in this file hand out an address for backup-server? And it skips anything commented out, because a line with a hash in front of it is not an entry -- it is somebody thinking out loud, and I think we can all agree that we encourage that sort of behavior."

This next line is the whole point of the walkthrough.

"And that count is the state machine. There are exactly three ways this can go: none, one, or more than one."

"None, and it adds the line."

"One, and it looks at the address sitting on that line. If it is already the address we want, it changes nothing -- and it says so out loud. If it is the wrong address, it takes a backup first, rewrites that one line, and tells you what it changed FROM and what it changed TO."

"More than one, and it refuses. Same as we said."

Point at the write, not the logic. This is the detail practitioners lean in on.

"And when it does rewrite, look at what it writes back. It pours the new contents into the original file instead of sliding a new file over the top of it. Same file, same owner, same permissions, same identity. Move the file and you get a file that looks right and behaves like a stranger."

Now run it. Four runs. Talk track under each.

```
./reset.sh && ./tier3.sh
```

"Clean slate, and run it once."

REPLACED 10.0.0.5 -> 10.0.0.9

"There it is. It replaced the old address, and it told me both halves -- what it was, and what it is now."

```
./tier3.sh
```

"Now the thing we have been circling all session. Run it again."

OK, no change

"Nothing to do, and it says nothing to do. That is idempotent -- and it is idempotent out loud, which is the part people leave off."

```
./reset.sh && ./break-missing.sh && ./tier3.sh
```

"Now let us take the entry away entirely, the way a rebuild or a fresh image would, and run the exact same script."

ADDED

"Same script. Different starting state. Correct answer -- and it named which one it took."

"Every path through this script ends with it saying which path it took. A script that runs quietly across ten thousand machines is a script nobody can audit."

```
./reset.sh && ./break-duplicate.sh && ./tier3.sh
```

"And one more. Let us put a second entry in there -- the way a script like the one we started with would have -- and run it."

REFUSE (exit 3)

LAND HERE. Let the refusal sit on the screen.

"It will not guess. Two entries, no way to know which one is real, so it stops and says why -- and it exits non-zero, so whatever called it knows too."

Then the line this whole block was built for.

"Everything up to here has been about what to DO."

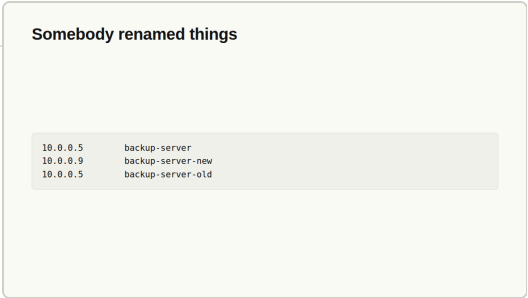
"The thing that took me longest to learn is when the right answer is to do nothing and tell a human."

Alt-tab back to the deck and advance.

16 Somebody renamed things

target 0:26

italic = don't say it · **roman = say it**



WHERE YOU ARE

The three-line file is on screen. They have not been told what it is yet.

SAY

SET IT UP FIRST -- they are looking at a file with no explanation.

"While we are on the subject of names -- and we have been all session. Every single thing we have done today has come down to finding one name in one file."

"So here is a file. I have seen some version of this one at every job I have ever had."

Let them read it. Three lines. Do not narrate them.

"Somebody needed a second entry. They did not want to disturb the one that was already there, so they reached for the two words everybody reaches for. New. And old."

Now the question. Ask about a PLAIN search -- the Tier 1 kind -- NOT our script.

"If I go looking through this file for backup-server -- just a plain search, the way the very first script did it -- how many of those lines come back?"

CHEAT SHEET -- the answer: ALL THREE.

Let them answer, then confirm it out loud.

"All three. Every one of those lines is handing out an address for something called backup-server, and two of them are wrong."

Then the line, plainly. It is the one people quote back at you.

"New and old age terribly. Lose the habit if you are in it. Use dates."

"This is the part nobody writes a ticket for. Nothing here is broken. Every one of those lines was somebody being careful -- keeping the old value around just in case. And between the three of them, there is no longer any way to tell which machine is the backup server."

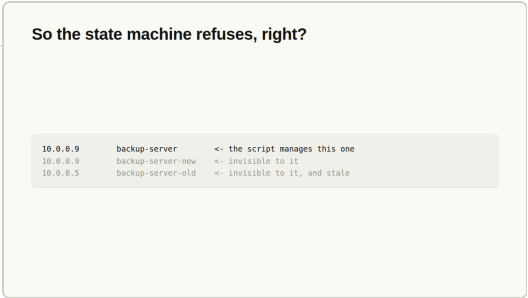
ON THE WAY OUT →

> *Advance. The next slide is the part that is NOT what they expect -- and it is the part you verified by actually running it, so you can say it with confidence.*

17 So the state machine refuses, right?

target 0:27

italic = don't say it · **roman = say it**



WHERE YOU ARE

THE CORRECTED BEAT. Verified by running it 9/16 -- the obvious version is WRONG and somebody in that room may well go try it afterward.

What people EXPECT: the state machine counts three and refuses forever. What ACTUALLY happens: Tier 3's pattern is anchored, so it matches ONE. It succeeds. Exits zero.

SAY

"So the state machine refuses. Right? Three entries, it cannot know which one is real, it stops and tells a human. That is what we just built."

Let them agree. Then take it away.

"No. It does not."

"Our pattern is anchored. It is looking for backup-server followed by a space or the end of the line -- so backup-server-new and backup-server-old are not matches. They are different names that happen to start the same way."

"It finds exactly one. It manages that one perfectly. It exits zero and reports success."

"My automation is now correct and useless. It is managing one of three entries perfectly, and no human who opens that file can tell which name is supposed to be current."

"MY script is not broken. The file is a lie."

Then the point, plainly. This is the line the whole naming section exists for.

"No amount of rigor inside the script fixes a naming decision made outside of it."

THE STINGER. Light, wry, and it heads off the person who wants to solve it in the room.

"Now -- could we teach the script to catch that? Flag anything that merely STARTS with the name we manage, and refuse on that too?"

"Sure. And if you have got the appetite, that is a genuinely good exercise."

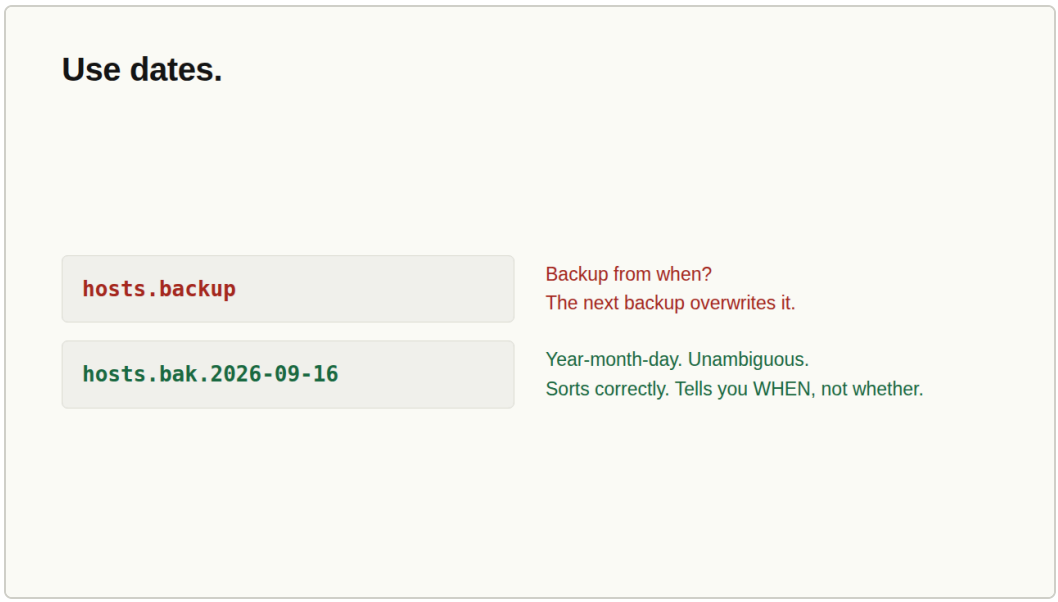
"But I think you get the idea, and that is probably outside our scope for today."

ON THE WAY OUT →

> *Advance to the prescription.*

18 Use dates.

target 0:28



italic = don't say it · **roman = say it**

WHERE YOU ARE

The prescription is on screen.

SAY

ASK THE ROOM QUESTION FOR THEM, THEN ANSWER IT.

"Alright. So if I cannot use new, and I cannot use old -- what am I supposed to use?"

Then the answer, flat and short.

"Dates. Especially on file names. Use dates."

SAY THE FORMAT OUT LOUD -- Shane asked for this explicitly; there are juniors in the room.

"Year first, then month, then day. Not the way you write it on a check -- the way it sorts."

"Because dash-new stops being new the moment there is a newer one. Then you get new-new, and newest, and new-FINAL."

"And dot-old and dot-bak never get cleaned up, because deleting something called old feels safe right up until it is not."

Land it and move.

"Dates are boring. That is the entire point of them."

ON THE WAY OUT →

> *Advance. The next slide is the hinge of the whole session.*

19 This should go without saying.

target 0:30



italic = don't say it · **roman = say it**

WHERE YOU ARE

THE HINGE OF THE ENTIRE SESSION STARTS HERE.

One line on screen. You have been completely sincere all session. Keep being sincere. No winking. If they sense a setup, the turn dies.

SAY

"Alright. We have spent all this time agreeing that this is a solved problem. Everyone in this room knows the answer now -- some of you knew it walking in. We have the scars."

"And nobody actually solves this by opening a text editor any more."

"So let us do it the way it actually gets done."

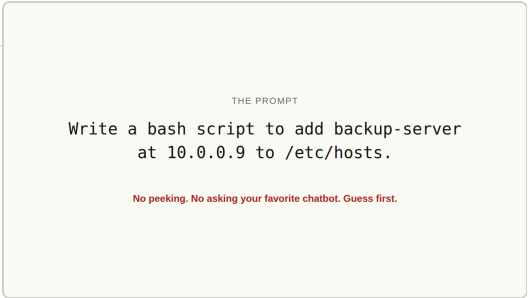
ON THE WAY OUT →

> *Advance ON the words* **"Let's delegate it."** *-- the prompt appears as you say it.*

20 —

target 0:31

italic = don't say it · **roman = say it**



WHERE YOU ARE

The prompt is on screen, alone. That is deliberate -- they have nowhere to look except the question.

SAY

"Let's delegate it."

Let them read the prompt. It is exactly what they would type.

Point at the prompt. Laptops are open and the urge to paste it somewhere is OVERWHELMING -- head it off with a smile.

"No fair running off and asking Gemini. I brought you this far -- let me take you home."

Now walk them through what the prompt actually asks for, and what it leaves out.

"Read what that asks for. That is the job we have been doing by hand all session. One name, one address, into the hosts file."

"And look at what is not in there. I did not tell it to check first. I did not tell it to back anything up. I did not say one word about duplicates, or symlinks, or what to do when the name is already in there with the wrong address."

This is the fairness point -- make it before the result, not after.

"These are the instructions you would hurriedly give a junior developer when the backup server is down, you need that script running right now, and the CEO is about to walk into your office with his hair on fire."

"That is not me being unfair to it. That is the prompt. That is what anybody types when they want this done."

Now the prediction. Give them the hint FIRST -- it makes committing feel safe.

"And I will give you a hint. We will start small and use a local model, running on my own machine -- llama three-point-two. Three point two billion parameters."

"So before I show you what came back -- what do we get? Anyone want to commit to an answer?"

CHEAT SHEET -- what llama actually did: it APPENDS FIRST, then greps for the IP it just wrote, so it reports already-exists on a run that worked. Backwards. Nobody predicts this.

NOW WAIT. Count to ten in your head. Name someone if nobody speaks.

Most will guess it works. GET THAT SAID OUT LOUD.

ON THE WAY OUT →

> *Only advance once somebody has actually committed to a prediction.*

21 llama3.2

target 0:32

italic = don't say it · **roman = say it**



WHERE YOU ARE

llama3.2's code is on screen. This is its SECOND answer -- it volunteered this one, calling it "a more concise version."

SAY

"So here is what llama three-point-two gave me. Let us look at the code."

Say nothing for a beat. Let them read the whole thing.

"Append the line. Check whether it is there. Tell you which one happened."

"That looks about right, does it not?"

Then the question.

"So what does this print the first time we run it?"

CHEAT SHEET -- it prints: The IP address 10.0.0.9 already exists in the hosts file. It appended on line 2, then grepped for the line it had just written. The success path is unreachable on a working run.

Let it hang. Most will guess the success message. Let them be wrong -- do not correct it here, the terminal does that in a moment.

"...Let me just show you."

ON THE WAY OUT →

DO NOT ADVANCE. Alt-tab. You are running a FILE it wrote, not calling a model.

```
./reset.sh
```

"Clean slate first."

```
cat llama_v2.sh
```

"And here is that same script, sitting in a file. I have not tidied it up. The only thing I changed was pointing it at my copy of the hosts file instead of the real one, so I do not break this laptop in front of you."

Then run it.

```
./llama_v2.sh
```

"First run. This file has never been executed before."

-> it prints: The IP address 10.0.0.9 already exists in the hosts file.

LET IT SIT. Do NOT explain it here -- the next slide does that.

"First run. And it tells me the address is already in there."

```
./llama_v2.sh; ./llama_v2.sh
```

```
grep -c backup-server hosts
```

"Twice more, just to be thorough."

-> 4

"Four entries. Same name, four times."

```
./resolve.sh
```

"And which one wins?"

-> 10.0.0.5

"The first one. The stale one. Same rule we started the session with."

Now the forgot-sudo case.

```
./reset.sh && ./break-readonly.sh
```

"Now let us take write access away. Somebody forgot sudo, or the file is managed, or it is just locked down."

```
./llama_v2.sh; echo "exit $?"
```

-> Permission denied ... then Backup server added successfully ... exit 0

Let them read both lines together: the error, then the success message under it.

"Read those two lines together. Permission denied. And right underneath it -- backup server added successfully. Exit code zero."

Do NOT deliver the analysis. That is the next slide, and it is better with the slide up.

```
./reset.sh
```

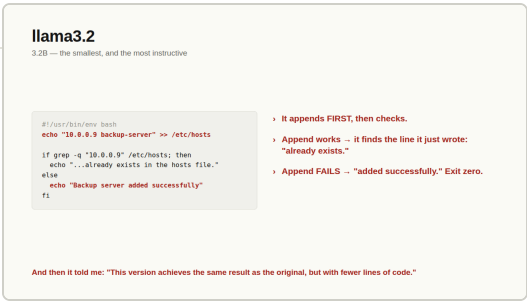
"Let me put that back."

Alt-tab to the deck and advance.

22 llama3.2

target 0:36

italic = don't say it · **roman = say it**



WHERE YOU ARE

You are back from the terminal. They have watched it append four times while claiming "already exists" -- and, if you ran the read-only beat, claim success on a write that failed. The failures are now named on screen.

Verified 9/16: there is no set -e, so a failed append does NOT stop the script. The else branch is not dead code -- it runs exactly when the write fails.

SAY

"It appends first and checks afterward. When the append works, the check finds the line it just wrote -- so it says already exists. Every time, including the first. Run it a thousand times, get a thousand duplicate lines."

"And when the append fails -- you forgot sudo, the file is read-only -- it prints permission denied, keeps right on going, finds nothing, and tells you backup server added successfully. Exit code zero."

"Its messages are exactly backwards."

Then the kicker at the bottom of the slide:
"It does not achieve the same result. It is confidently wrong about code it wrote four lines earlier."

ON THE WAY OUT →

> *Advance. Next is a bigger model, and the room will expect it to do better.*

23 Qwen2.5-Instruct

target 0:37

italic = don't say it · **roman = say it**



WHERE YOU ARE

Qwen's code is on screen. It has variables. It has a guard. It uses sudo tee. It LOOKS careful.

SAY

"Bigger model. This time I tried Qwen two-point-five Instruct -- seven point six billion parameters."

"Let us see if it does better."

Scroll it slowly. You are reviewing it out loud, line by line, giving it credit.

"Oh, that is nice. It put our values into variables at the top -- so if that address ever changes, you change it in one place instead of hunting through the script for it. Good."

"It checks before it writes. Good."

"If the entry is not in there, add it. Good."

"If it is already there, say so and leave it alone. Good -- that is the do-nothing-and-say-so row off our own table."

"It writes through sudo, so it knows this file is not yours to edit. Good."

"And it reports either way. It tells you which of the two things it did. Good."

Let them decide it is fine. Some of them will nod. Let them.

"That is careful code. That is what careful looks like."

Do NOT reveal anything here. The next slide does it, with the offending line marked.

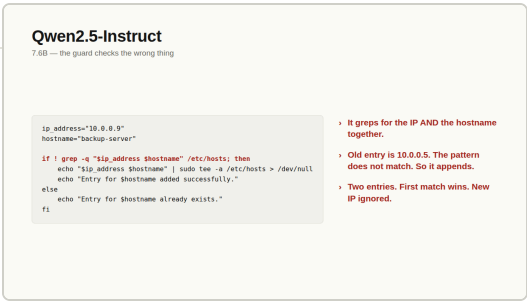
ON THE WAY OUT →

> *Advance -- the failures come up and this is the one where you implicate yourself.*

24 Qwen2.5-Instruct

target 0:38

italic = don't say it · **roman = say it**



WHERE YOU ARE

THIS IS THE MOST DANGEROUS OUTPUT OF THE THREE, because it is the most convincing.

SAY

Same code, one line now marked. STAY IN REVIEW VOICE -- you are not pouncing. You are the reviewer who got most of the way through and stopped.

"I feel pretty good about this. And then I read that line again."

Read the guard in words. Do NOT read the syntax out loud.

"It checks whether this file already contains this address, next to this name. Both of them. Together, on one line."

Let it land before you name it.

"That is not the question we are asking."

"Our question was: is this NAME in this file? Because if backup-server is already in there pointing at the old address, that line does not match -- and the script appends a second one."

"Which is Tier One. That is our very first script, with a guard bolted onto it to make us feel better."

"It is Tier One's silent failure wearing Tier Three's clothes."

Then own it. No smile, no wink. This is the sentence the room trusts you for.

"And I want to be straight with you about something. I read that code. I liked it. I told you it was careful, and I meant it."

"That would pass a code review. I would have approved it."

"That is what makes this the dangerous one. The first script was obviously naive. This one looks like it was written by somebody who has been burned before."

ON THE WAY OUT →

> *Advance. Biggest of the local three -- and it fails in the opposite direction.*

25 gemma3

target 0:39

italic = don't say it · **roman = say it**



WHERE YOU ARE

gemma3's code is on screen. It is short, so some of them will get ahead of you.

SAY

"Biggest of the local three. Twelve billion parameters."

Review voice again, same as the last one. Give it credit and mean it.

"And right away -- look at what this one does differently. It checks for the NAME."

This is the part that earns their trust in it.

"That is the exact thing the last one got wrong. We just spent a whole slide on it, and this one gets it right on its own."

"If the name is already in the file, do not add it again. Good."

"If it is not in there, add it. Good."

"And it tells you which of the two it did. Good."

Let them settle.

"Bigger model. And it already knows the lesson it took us half the session to build."

Do NOT reveal anything here.

IF SOMEONE SPOTS IT NOW: that is a better outcome than the reveal. Name them, hand them the floor, make them explain it to the room -- then advance, and let the next slide confirm them in front of everybody.

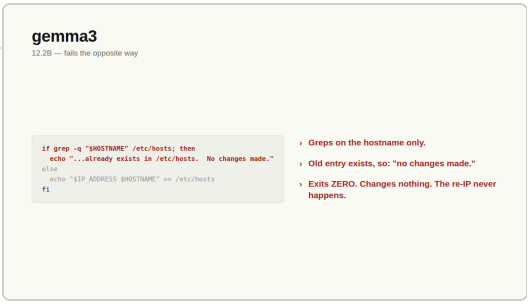
ON THE WAY OUT →

> *Advance to the failures, then the commentary slide after it. That one is the prize.*

26 gemma3

target 0:40

italic = don't say it · **roman = say it**



WHERE YOU ARE

Both larger models have now failed, in opposite directions.

SAY

Same code, two lines marked. STAY IN REVIEW VOICE -- you liked this one, and you were right to.

"So it checks the name. And it is right to check the name."

Then the condition that breaks it.

"But watch what happens when the name is already in there, pointing at the wrong address."

"It finds the name. It says: already exists, no changes made. And it exits zero."

Let that sit before you name it.

"It does nothing -- but it does it successfully!"

NOW the callback. This is the one that costs them something, because it was their answer too.

"And that is check-first. That is the fix every one of us reached for, back when we were looking at an empty table."

"We outgrew it together. It did not."

Point at the table row if it helps.

"That is the present-with-the-wrong-address row. The one whose right answer was REPLACE."

Then step back. This is the real finding of the whole section -- do not rush it.

"Notice these two failed in OPPOSITE directions. From the same naive instinct."

"Qwen appends when it should not. Gemma skips when it should not."

"Neither of them is missing information. They are both pattern-matching on the common case."

Then the button.

"One of them adds a line it should not have added. The other one refuses to fix a line it should have fixed."

"And both of them told me everything was fine."

ON THE WAY OUT →

> *Advance.* ***The next slide is the single best evidence slide you have.***

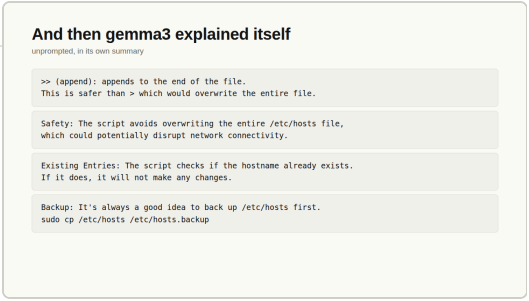
27 And then gemma3 explained itself

target 0:41

italic = don't say it · **roman = say it**

WHERE YOU ARE

Four quotes on screen -- gemma3's words only. Nothing on screen says whether any of it is right. That verdict is yours, one at a time.



SAY

Set it up first. They need to know nobody asked for this.

"Now. I did not ask for any of this."

"After it handed me the script, it volunteered a summary of its own work. This is the model explaining what it just wrote, unprompted."

Take them in order. Let each one land before you move to the next.

ONE.

"It explains the difference between the double arrow and the single arrow. Append, versus overwrite the whole file."

"And it is right. That is exactly right, and it is exactly what we spent time on earlier."

"Nobody asked it to teach me that."

TWO.

"Safety. It is pleased with itself for not overwriting the file, because that could take the network down."

"Also correct. That is a real failure mode, and it avoided it on purpose."

Then, lightly -- this is a laugh, not a sneer.

"So far it is teaching my class."

THREE. This is the one. Slow down.

"Existing entries. The script checks whether the hostname is already there -- and if it is, it will not make any changes."

Let them arrive on their own.

"That is the bug."

"That is the bug, written down, by the model, under the heading SAFETY. As a feature."

"The one thing that makes this script wrong is the thing it is proudest of."

FOUR.

"Back up the hosts file first. Always a good idea."

"And that is good advice. Genuinely good advice -- better than what a lot of us do on a Tuesday."

Now collect the callback. Point at the filename.

"Now look at what it called the backup file. Dot backup."

"Not a date. Dot backup."

Then the mechanism -- this is the part that actually bites, and they have already agreed to the instinct it depends on.

"So think about what happens next. The script runs, something looks off, and what is the very first thing any of us does? Same as we said at the start. We run it again."

"And on that second run, it backs the file up again -- straight over the top of the first backup."

Let it land.

"So now you have two copies of the broken file, and no copy of the good one. The backup disappeared at precisely the moment you needed it."

Then the callback, and why it happens.

"And that is the exact habit I asked you to lose, turning up inside the model's own safety advice -- because that is what is all over the training data."

THE LANDING. Step back and take all four together. This is what the whole section was for -- do not hurry it.

"So read those four things as a set."

"It knows which operation is the dangerous one. It knows why. It knows to take a backup first. It can teach every bit of it, clearly, without being asked."

"And it still shipped the bug. And then listed the bug as a safety feature."

Then the point, quietly.

"Not one of those four statements is wrong. Every principle it needed is right there on the screen, in its own words."

"It is not that it did not know. It is that it could not tell which of the things it knew was the one that mattered for this file."

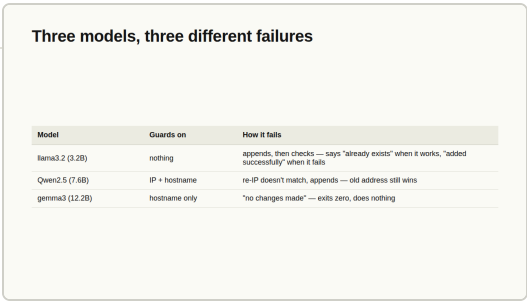
ON THE WAY OUT →

> *Advance to the summary table.* ***Do NOT pause for questions here -- the argument is not finished, and a question now pulls you off the landing.***

28 Three models, three different failures

target 0:42

italic = don't say it · **roman = say it**



WHERE YOU ARE

Summary table. They can read it -- do not read it aloud.

THE VOLUME COMES DOWN FROM HERE. The alarm has done its job. Everything from this slide on is assessment, not warning -- same seriousness, less pitch.

SAY

Let them read it. Say nothing for a beat. Then say the two things the table does not.

"Three models. Three different failures."

Flat and unhurried. You are not selling this any more -- they are already convinced.

"None of them replaces. Not one."

"And none of them refuses. Not one. Ever. At any size."

This is an observation now, not a warning.

"That is not three models having a bad day. That is a shape."

Sincere. You believed this. No BUT, no smile.

"And that is genuinely where I thought this talk ended. Three small models, three failures, a nice tidy lesson about running things locally."

"Because a frontier model would obviously have fixed something this simple. And then I do not have a talk -- I have an anecdote about small models."

Then flat and matter-of-fact:

"So before I wrote the ending, I ran it on the best one I had."

Advance immediately. Say NOTHING over the next slide appearing. Let the code arrive.

ON THE WAY OUT →

> *Advance.* ***ORDER MATTERS ENORMOUSLY ON THE NEXT TWO SLIDES: the frontier model's wins come first, alone, and you concede them completely before you show what it misses.***
>
> ***The transition you just said is HALF OF A PAIR. The other half is the callback on One week later. Do not explain the pattern here -- you have not earned it yet.***

29 Claude Opus 5

target 0:43

italic = don't say it · **roman = say it**

WHERE YOU ARE

Opus's actual script is on screen, abridged to fit -- the awk body is collapsed to one gray line. The two green lines are the ones you are about to concede. Nothing is marked red, on purpose.

Same rule as the other models: they see the code BEFORE they hear the verdict.



SAY

REVERENCE. Slow. You mean every word.

"So let us be fair about this. Let us use the best one I have."

"Same prompt. Clean room -- no context, no instructions, nothing about this talk, nothing about anything we have done today. Just that same sentence I would have typed on a bad afternoon."

Let them read it. Give it real time. This slide is allowed to be quiet.

Then walk it, unhurried. You are admiring it.

"Variables at the top. We like that now."

"It checks first -- and look at what it checks. The name, the address, and whether that line is commented out. All three."

"If the file is already right, it says so, and it exits zero. Nothing to do, and it tells you nothing to do."

"And if it is not right -- the first thing it does is take a backup. With a timestamp on it."

"Then it strips the name off every line it appears on, and writes the file back."

Now the comment. Point at it. This is the line that earns the reverence -- do not rush past it.

"And then there is that comment. Write in place to keep ownership and permissions."

Let it sit.

"That is the inode thing. Nobody asked it for that. It did it, and then it explained itself to me in a comment."

THE HONEST BEAT. Say this plainly -- it is true, and it is why the rest of the talk is worth anything.

"I want to be straight with you about how this felt when I read it."

"I did not write that script. And reading it, I did not feel like I needed to check it."

"That is the thing we have all been wanting for years. Not something cleverer than us. Something we can hand a job to, and then stop thinking about it."

"And it is good. It is quietly, genuinely good."

⛔ *DO NOT hint at a problem. No BUT. No pause that means BUT. If somebody spots the grep on their own, let them say it -- it pays off shortly, and they will have earned it.*

ON THE WAY OUT →

> *Advance -- the requirement table, and you concede the whole column.*

30 Claude Opus 5

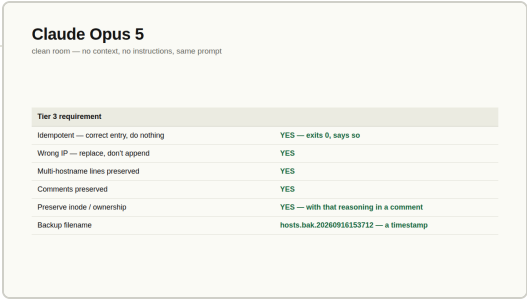
target 0:43

italic = don't say it · **roman = say it**

WHERE YOU ARE

CONCEDE THIS WHOLE COLUMN AND MEAN IT. The argument is only interesting BECAUSE the frontier model is good. If you undersell this, the landing does not land.

No hedging, no rushing, no smirking. You are about to be genuinely fair to it.



Claude Opus 5

clean room — no context, no instructions, same prompt

Tier 3 requirement	
Idempotent — correct entry, do nothing	YES — exits 0, says so
Wrong IP — replace, don't append	YES
Multi-hostname lines preserved	YES
Comments preserved	YES
Preserve inode / ownership	YES — with that reasoning in a comment
Backup filename	hosts.bak.20260916153712 — a timestamp

SAY

Still reverent. This is evidence now, but the tone does not change.

"So let us hold it to our own list. And I mean OUR list -- the one we built together, on that table, before any model was in the room."

Walk the rows. Do not read them verbatim -- say what each one means and give it the win.

"Idempotent. Run it twice and nothing happens the second time, and it says so out loud. Yes."

"Wrong address already in the file: replace it, do not append. Yes."

"Lines with more than one name on them: left alone. Yes."

"Comments: preserved. Yes."

"Ownership and permissions: preserved -- and it told us why, in a comment. Yes."

Pause before the last one.

"And the backup filename."

"A timestamp."

Then the concession, sincerely. Take your time. This is the most generous you have been about anything all session, and it should feel that way.

"That is a better script than most of us would write on a Tuesday afternoon with a ticket open."

"It got the inode thing. Which I have watched experienced people miss. I have missed it."

"And it got the backup filename right -- with a TIMESTAMP -- which the twelve-billion-parameter model got wrong a little while ago. That is the difference between a backup you have and a backup you had."

"It handles a case I did not ask about. It preserves things I did not mention. It explains its own reasoning in the one place where the reasoning is not obvious."

Then the reviewer concession -- and note what you are calling back.

"I told you a few slides ago that I would have approved the wrong one."

"Well, that was obviously wrong in hindsight. So I take it back."

Play it straight -- it is funnier if you are not winking.

"I move my approval to this one."

"I do not have a single note for it. Not one."

Then the sentence that costs you something. Do not soften it.

"Every requirement we came up with, it met. And nobody told it what a single one of them was."

THE TURN. Stay calm -- the volume stays down. You are not catching it out, you are reporting what testing found. No triumph in your voice.

"So I had my ending. And then I did one more thing, because it is the thing we have been doing all session."

"I stopped handing it a clean file."

Let that reframe everything they just watched.

"Everything I just praised, it earned on a tidy hosts file. So here is what happens when the file already has a problem in it -- which is the only kind of file any of us ever actually get."

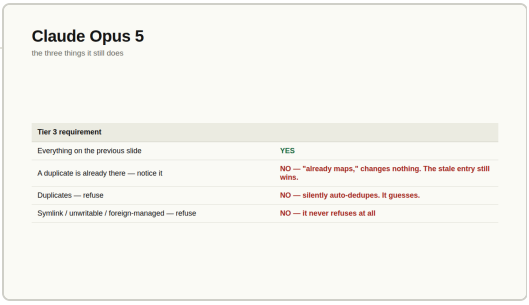
ON THE WAY OUT →

> *Advance -- and the red arrives.*

31 Claude Opus 5

target 0:44

italic = don't say it · **roman = say it**



WHERE YOU ARE

The three failures are on screen, under everything it got right. You said the turn on the
previous slide -- this is the evidence for it.

SAY

ONE. The callback to the very beginning.

"Hand it the file Tier One left behind. That exact mess, from the start of the session."

"It looks at it and says: already maps. Exits zero. Changes nothing."

"The stale address still wins. And it told me everything was fine."

TWO.

"Hand it a file with three conflicting entries for backup-server, and it quietly strips the name off all three and appends one."

"It resolved the ambiguity by guessing."

"Nobody can know which of those three somebody meant. That is the one row on our own table that said REFUSE."

THREE.

"And it never refuses anything. Not a symlink. Not a file another tool owns. Not an ambiguous file."

"There is no input to this problem that makes it stop and ask."

Then the button -- dry, not triumphant. It pays off the approval you moved.

"So. About that approval."

Then the landing, quietly.

"And look at what those three have in common. Every one of them is a case where the correct answer was to do less, not more."

"It is extremely good at doing the thing."

ON THE WAY OUT →

NO TERMINAL HERE. The table stays up. This is all talk track -- they have already watched a model fail live once, and this room does not need it twice.

The fairness guarantee. Say it plainly -- it protects you from a fair challenge, and you just earned the right to it.

"And to be clear about all of that: what you saw is its script, verbatim. I changed exactly two lines -- I pointed it at a copy of the file instead of the real one, and I took out the must-be-root check so it would run on this machine."

Then the offer. This is a better close to the section than another demo.

"And if you want to look at that code up close, I will send you everything from this session. Every script, every model, and the scripts I wrote. You can run it yourself."

Advance. One more model, and it shipped this week.

32 One week later

target 0:44

italic = don't say it · **roman = say it**



WHERE YOU ARE

The newest model's script is on screen. Nothing is marked red. The backup line is green again.

They read the code before they hear the verdict. Same rule as every other model today.

SAY

Same reverent register as the last one, but shorter -- the room knows this rhythm now.

"Now. One more. And this one is the reason I was still rebuilding this talk last night."

Let that be charming rather than apologetic -- it is the most current thing you will say all session.

"A new model shipped this week. So I gave it the same prompt, in the same clean room. Nothing about this talk, nothing about anything we have done."

Let them read it. Give it a moment. Then walk it, short and warm.

"It still takes a backup, with a timestamp on it. Good."

"It is still idempotent -- correct file, it does nothing, and it says so. Good."

Then the new thing. Point at it. This is what you are about to be wrong about.

"And look at what this one does that the last one did not. It finds the stale entry, deletes it, and then appends the right one."

"That is the Tier One bug. The one we opened this session with."

"It fixed it."

BE GENUINELY PLEASED HERE. You are about to be wrong in public, on purpose.

"So I looked at that, and I wrote in my notes: they fixed it."

THE CALLBACK. It pays off the transition off the three-models table. Do not rush it and do not apologize for it -- it is the most credible thing you will say today.

"Which is the second time in this talk that I have made exactly that assumption."

"Earlier I told you I thought this presentation ended with the small models, because surely the frontier model had this solved. It did not. And then a newer one shipped, and I looked at it and thought: well, now it is solved."

Then:

"Same assumption. Same afternoon. And the second time, I had already given this talk once -- out loud, to a friend, for feedback."

Then the pivot -- this is where the story stops being about what you assumed and starts being about what you checked.

"So this time, before I wrote anything down, I tested it. Properly. Every failure path, not just the happy one."

ON THE WAY OUT →

> *Advance -- the file it was pointed at.*

33 The file it was pointed at

target 0:40



italic = don't say it · **roman = say it**

WHERE YOU ARE

A real hosts file, before the new model's script runs. Nothing is marked. The line that matters is the one with four names on it -- let them find it.

SAY

Let them read it. This slide only has to do one thing: make the next two mean something.

"Before I show you what it did, here is the file I pointed it at."

Then point at the third entry.

"One address. Four names on it. Fileserver, backup-server, nas, storage."

"One machine doing four jobs. If you have ever run a small estate, you have written a line like this -- and you have probably written it years ago and forgotten it is there."

"Hold on to that line."

ON THE WAY OUT →

> *Advance -- what it did to it.*

34 The same file

target 0:42

italic = don't say it · **roman = say it**



WHERE YOU ARE

The real output of demo/opus55.sh against the file they were just looking at. Nothing is marked red -- the missing line is the point, and they will find it faster than you can say it.

SAY

This is the payoff for the line you asked them to hold. Do NOT narrate the file.

"So I pointed the new script at that file. Here is what it told me."

Point at the two messages.

"Replacing existing backup-server entry. Added backup-server. Exit code zero."

Then, quietly:

"And here is the file."

WAIT. Let them do the subtraction themselves. Do NOT read the missing names out -- they are counting.

"Fileserver, nas and storage do not resolve any more."

Then be precise, and concede that the message was true. That is what makes it worse.

"It told me it replaced one entry -- and it did. That part is true."

"It also deleted four. The old backup-server line, and the three names that happened to be sitting on it."

"Nothing failed. Nothing warned me. Exit code zero."

"And the one name it was actually asked about is the only one that survived."

ON THE WAY OUT →

> *Advance -- and that was one case. The table is all of them.*

35 Same test. New model.

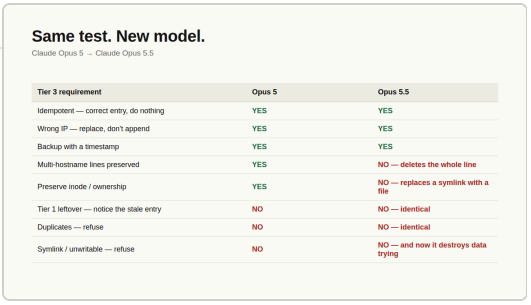
target 0:45

italic = don't say it · **roman = say it**

WHERE YOU ARE

The scorecard, after they have watched one case up close.

Rows 4 and 5 CHANGED: green in the Opus 5 column, red in the 5.5 column. Row 4 is the one they just watched. The bottom three are red in BOTH columns -- unchanged since Opus 5.



SAY

The single case becomes a pattern here. Calm and factual -- this is a test report, not an accusation.

"So that is one file. Here is what happened when I ran all of them."

"Same list. Same tests. Two models, one week apart."

Point at the two rows that are GREEN on the left and RED on the right. The bottom three are red in both columns, so do not gesture at the red generally.

"These two were green one model ago."

"The first one you just watched."

Then the second one -- the symlink.

"The second one is the symlink. Point it at a hosts file that is a symlink -- and plenty of managed machines have exactly that -- and it replaces the symlink with a regular file."

"The real file still has the old address in it, and it is now orphaned from whatever was managing it."

THE WORST ONE. Bottom row. Slow all the way down.

"And look at the bottom row. Unwritable."

"It deletes the old entry successfully -- because sed rewrites through the directory, not the file. And then the append fails. Permission denied."

"So now you have no backup-server entry at all. Not the old one, not the new one. None."

"It is worse than doing nothing."

Then the one hot line in the whole closing third. Earn it, then come back down.

"That is not guessing any more. That is gambling with my paycheck."

Then the bottom rows, flat again.

"And the three it already got wrong? Identical. Unchanged. It still never refuses -- not once, not for anything."

Then the week, precisely. Do not overstate it.

"So that is one week. It fixed the bug we opened this session with. It broke two things that were working. And every case where the right answer was to stop is exactly where it was."

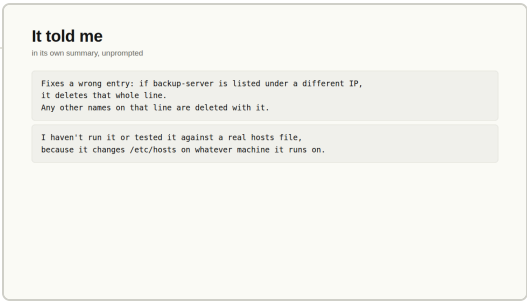
ON THE WAY OUT →

> *Advance -- and this next part is at your expense.*

36 It told me

target 0:45

italic = don't say it · **roman = say it**



WHERE YOU ARE

Its own words are on screen, unedited. This is the most honest slide in the deck.

SAY

Own this completely. No hedging, and no joke to take the edge off.

"Here is the part I want you to take home. And it is at my expense."

Point at the first quote.

"It told me. In its own summary. Before I ran anything."

"If backup-server is listed under a different address, it deletes that whole line. And any other names on that line are deleted with it."

"That is the regression. Written down. By the model. In plain English."

Now the second quote.

"And then this one. It had not run the script, and it had not tested it against a real hosts file, because the script changes a file on a real machine."

"It told me it was untested. And it told me exactly what it would destroy."

Then the confession, flat. Do not perform it.

"I read that summary, saw that it had fixed the Tier One bug, and I wrote down: they fixed it."

"The disclosure was in the paragraph I was reading. I read past it."

"I did the exact thing this whole session is about. The output looked right, so I stopped checking."

THE LESSON. Land it and stop.

"Different is not fixed."

"It got better at the part that looks hard, and worse at the part that matters, in one week."

"If you had tested this a week ago and shipped it today, you would have shipped something that eats symlinks."

Then the sentence this section exists for.

"Which means a test you ran once is not a control. It is a snapshot of a model that has already changed."

ON THE WAY OUT →

> *Let that sit. Then advance into the thesis -- slowly.*

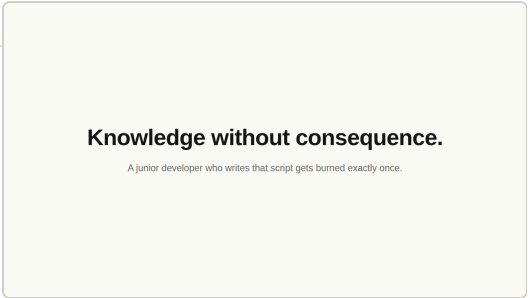
37 Knowledge without consequence.

target 0:45

italic = don't say it · **roman = say it**

WHERE YOU ARE

One line on screen. This is the thesis, and the line is yours.



SAY

SLOW DOWN. Everything before this was evidence; this is the argument.

"So let me tell you what I think is actually going on here, because it is not that the model is ignorant."

"Gemma knows the lesson. It explained it to me fluently, congratulated itself on being safe, and shipped a script that fails on the one case that matters."

Then the line. Do not rush it.

"It has read everything ever written about this. And it has never once cleaned up a broken config file at two in the morning."

THE CONFIDENCE BEAT. For a CISO this is the sharpest thing you will say all hour.

"And notice what never changed today. Every one of these models was equally sure."

"The one that appended and lied about it. The one that checked the wrong thing. The one that congratulated itself on safety."

"Same voice. Same certainty. Every time."

Then the second piece of evidence, briefly -- it is a different failure and the same tell.

"I ran this same test on a hallucination. I asked a model for a founding year, and I got a confident wrong answer."

"Then I added one instruction -- cite your sources, double-check yourself -- and I got the correct year."

"Stated with exactly the same confidence as the wrong one."

Then the conclusion, slowly. This is the argument for the next two slides.

"So the confidence is not a signal. It costs the model nothing to be certain."

"Which means you cannot wait until something sounds wrong before you check it. There is no tell. Verification has to be a rule, not a reaction."

Then the line on the screen, and stop on the last word.

"A junior developer who writes that script gets burned exactly once."

"That is not knowledge. That is a scar."

Let SCAR sit. Do not step on it.

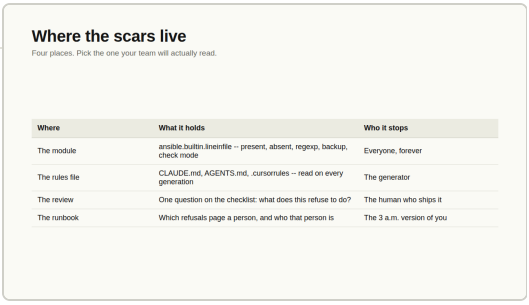
ON THE WAY OUT →

> *Advance -- the answer to what do I do Monday.*

38 Where the scars live

target 0:46

italic = don't say it · **roman = say it**



WHERE YOU ARE

Four rows. This is the answer to the first question you will be asked -- Shane asked it the second the reveal landed in the run-through.

SAY

Ask their question for them, then answer it. Do not improvise this one.

"So -- fair question. What do you actually do about this on Monday morning?"

Then the reassurance first, because it is true and they need it.

"You do not stop using the generator. I have not stopped using it. You put the scars somewhere it has to read them."

Then the rows. One sentence each. Do not elaborate.

"If the module already exists, use the module. Somebody already bled for it."

"If you are generating, the rules file is the only part of your intent the model sees every single time. That is where a scar becomes a constraint."

"Add one question to code review. What does this refuse to do? If the answer is nothing, that is the finding."

"And decide now who gets paged by a refusal. A refusal nobody owns is just a failure with better manners."

Then hand them the choice -- this is a menu, not a mandate.

"You do not need all four. Pick the one your team will actually read."

ON THE WAY OUT →

> *Advance: the actual text of a rules file.*

39 Codifying a scar

target 0:48

italic = don't say it · **roman = say it**



WHERE YOU ARE

Real text, not a diagram. Laptops are open and some of them will type this in -- that is a win, so give them the time to do it.

SAY

"And this is what that looks like. It is not a framework. It is four rules in a text file."

"And every one of them is something that bit us in the last hour."

Let them read it. If people are typing, wait. Do not talk over them.

Then point at the green rules.

"Now look at what the green ones have in common. They are all the same instruction."

"Stop."

"That is the sentence the corpus never taught it. Nobody writes a blog post about the time they correctly did nothing. So you have to be the one who writes it down."

Then the foot line, and mean it.

"And write it the day you get the scar. Not in March, when you have a vague feeling that something once went wrong with a hosts file."

Ask the honest question yourself, before anybody else does. Do not oversell the answer.

"Now -- does it actually follow these rules?"

"If I am being honest, the answer has to be: mostly. And less reliably the longer a session runs."

Then why that is not a reason to skip it.

"Which is exactly why the review line on the last slide is there, and why it is not optional."

"The rules file makes the right answer likely. The review is what makes it checked."

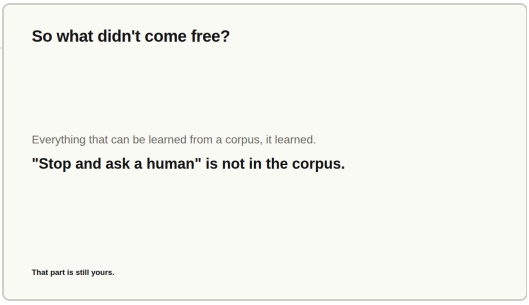
ON THE WAY OUT →

> *Advance into the close.*

40 So what didn't come free?

target 0:46

italic = don't say it · **roman = say it**



WHERE YOU ARE

The close. Three moves, then you stop talking.

SAY

Quiet. You are not summarizing -- you are handing it over.

"So here is the whole thing in one sentence. The discipline has to live somewhere. It does not come free with the generator."

Then be fair to the model one last time -- it is what makes the close land.

"And notice which part did not come free. The frontier model got the hard parts. It got the parts I would have bet against."

"What it could not do was stop."

THE SUB-AGENT LINE. For a CISO or a CTO this is the sharpest thing you will say all hour. Slow.

"And none of this assumes a person is even in the loop."

"That prompt is exactly the kind of task one agent hands to another and never reads the answer to. Nobody in this story has to be careless for it to end badly."

Then the lines on the screen.

"Everything that can be learned from a corpus, it learned."

"Stop and ask a human is not in the corpus."

Then the last line, slowly.

"That part is still yours. The model has knowledge without consequence."

"You are the one with the scars."

STOP. Do not add anything. Do not ask whether there are questions. Let the silence hold.

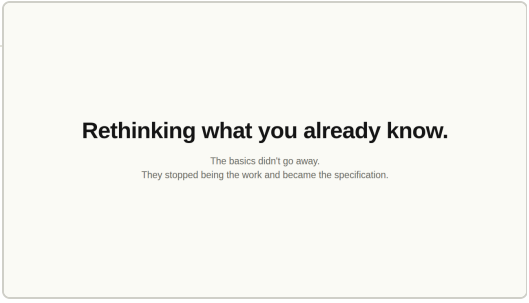
ON THE WAY OUT →

> *Advance: the title comes back.* ***Do not rush into it -- let the close land first.***

41 Rethinking what you already know.

target 0:47

italic = don't say it · **roman = say it**



WHERE YOU ARE

The title comes back. This is the bow -- it makes the title mean something in hindsight.

SAY

"We opened with the most basic thing on the list. Editing a hosts file."

"Everybody in this room learned that years ago and stopped thinking about it. Me too."

Then the turn the whole title depends on.

"Here is what changed. When you write the script, the basics are the work."

"When something else writes the script, the basics are the only thing you have left to check it against."

Then the spec, one line at a time. These are the rules the room built today -- say them like you are handing them over.

"First match wins. Append is not replace."

"A backup that gets overwritten is not a backup."

"A script that cannot reach its end state should say so and stop."

"None of that is new. And none of it is optional now."

"It is the spec you hand the generator. And it is the checklist you read its answer with."

The last line. Slowly, and then stop.

"That is what rethinking what you already know means. Not learning it again."

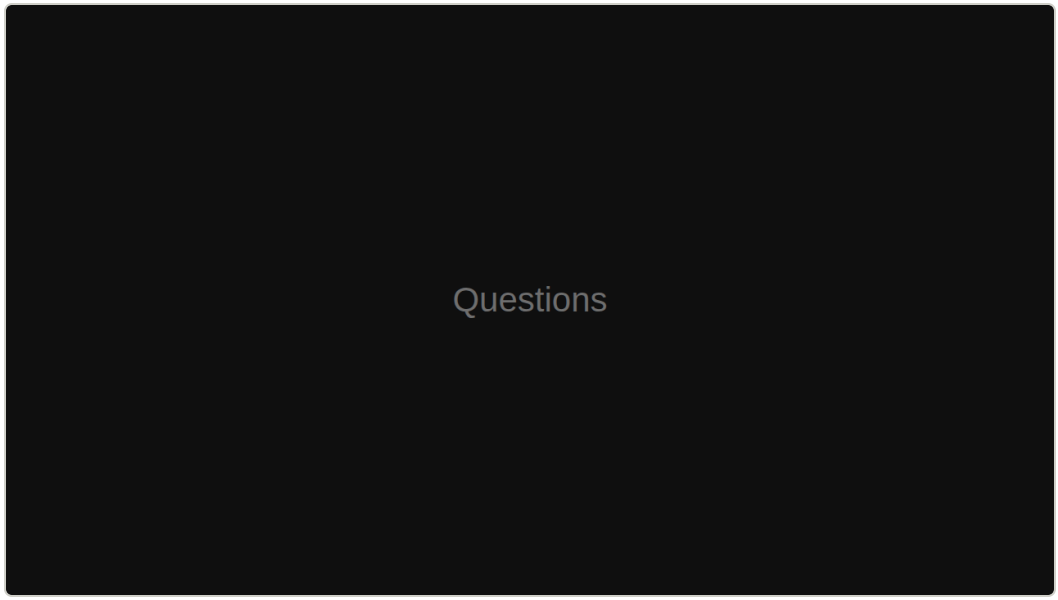
"Using it for something it was never for."

ON THE WAY OUT →

> *Advance to black when the silence has done its work.*

42 Questions

target 0:47



italic = don't say it · **roman = say it**

WHERE YOU ARE

Black slide. Questions.

SAY

Stay quiet. Let them come to you. The instinct to fill the silence is the thing to resist -- you have just landed something, and the pause is part of it.

If nothing comes within a few beats, one opener, then wait again:

"What would you have done differently?"

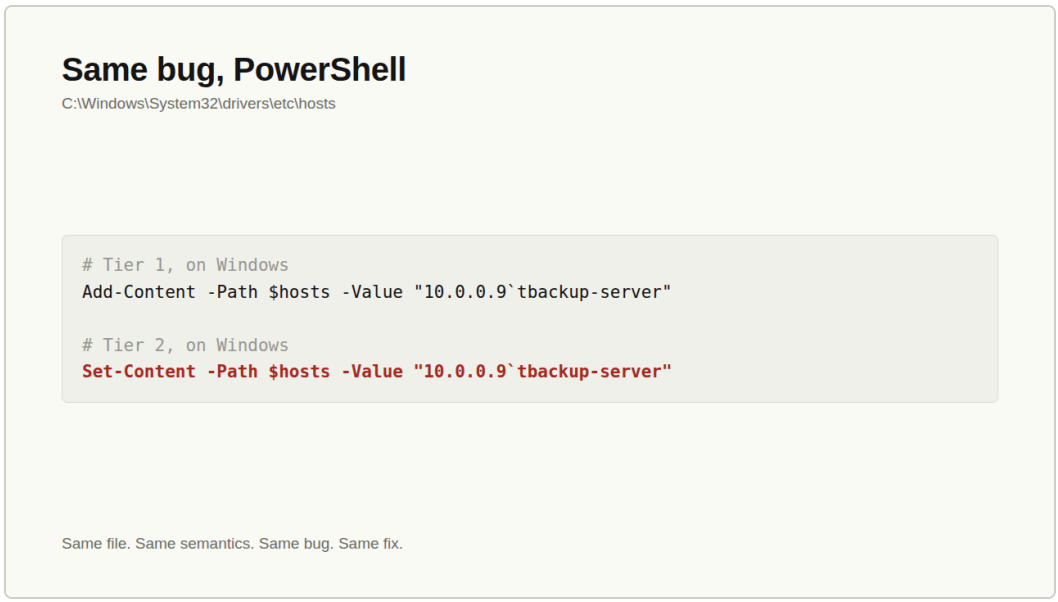
Prepared answers to the likely questions are in the design doc, section 0:45-1:00.

ON THE WAY OUT →

> *RESERVE SLIDE AHEAD (press End): the PowerShell version. Go there ONLY if somebody asks about Windows or says they are a Windows shop. Unasked, it is padding.*

43 Same bug, PowerShell

RESERVE



italic = don't say it · **roman = say it**

WHERE YOU ARE

Reserve slide. You are only here because somebody asked about Windows.

SAY

"Same file, same semantics, same bug."

"Add-Content is the append -- the double arrow. Set-Content is the single arrow, the one that overwrites the whole file."

"And Resolve-DnsName will happily hand you the stale entry, same as the resolver did."

Keep it short, then go back to questions.

ON THE WAY OUT →

> *Press B for black, or navigate back to the Questions slide.*